```
### Sun-Earth L1 Transfer — 3D CR3BP, Minimum-Energy, uz = 0
### Based on: Mini magic notebook
### Change from original: objective switched from minimum-fuel (L1 norm)
###                       to minimum-energy (tf-scaled L2 norm squared)

### Cell 1: Imports
begin
    using Pkg
    Pkg.activate(".")
    Pkg.instantiate()
    using InfiniteOpt, Ipopt, Plots, NLsolve, LinearAlgebra
end

### Cell 2: Parameters (Sun-Earth, 3D but uz = 0)
```

```
  Activating project at `~/.julia/pluto_notebooks`
```

```
begin
    μ           = 3.00348961491547e-6   # Sun = primary, Earth+Moon = secondary
    const u_max      = 0.1
    const N_supports = 501
    tf_lower    = 50.0
    tf_upper    = 400.0
    tf_start    = 200.0
    println("μ = ", μ, ", u_max = ", u_max, ", supports = ", N_supports)
end

### Cell 3: Compute L1
```

```
μ = 3.00348961491547e-6, u_max = 0.1, supports = 501
```

```
0.9900265839144706
```

```julia
begin
    function dOmega_dx(x, μ)
        r1 = abs(x + μ)
        r2 = abs(x - (1 - μ))
        return x - (1 - μ) * (x + μ) / r1^3 - μ * (x - (1 - μ)) / r2^3
    end

    res = nlsolve(x -> dOmega_dx(x[1], μ), [0.99])
    if !NLsolve.converged(res)
        @warn "L1 solver failed"
    end
    x_L1 = res.zero[1]
    println("Sun-Earth L1 at x = ", x_L1)
    x_L1
end

### Cell 4: Model + variables (full 3D states, only ux/uy control)
```

```
Sun-Earth L1 at x = 0.9900265839144706
```

```
An InfiniteOpt Model
Feasibility problem with:
  Finite parameters: 0
  Infinite parameter: 1
  Variables: 10
  Derivatives: 0
  Measures: 0
  `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.LessThan{Float64}`: 3 constraints
  `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.EqualTo{Float64}`: 1 constraint
  `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.GreaterThan{Float64}`: 3 constraints
Names registered in the model: tf, ux, uy, uz, vx, vy, vz, x, y, z, τ, ∂Ωx, ∂Ωy, ∂Ωz
Transformation backend information:
  Backend type: TranscriptionBackend
  Solver: Ipopt
  Transformation built and up-to-date: false
```

```julia
begin
    model = InfiniteModel(Ipopt.Optimizer)
    set_optimizer_attribute(model, "max_iter", 5000)

    @infinite_parameter(model, τ ∈ [0, 1], num_supports = N_supports)
    @variable(model, tf_lower <= tf <= tf_upper, start = tf_start)

    # Full 3D states
    @variable(model, x,  Infinite(τ), start = τ -> (1 - μ + 0.001) * (1 - τ) + x_L1 * τ)
    @variable(model, y,  Infinite(τ), start = τ -> 0.0)
    @variable(model, z,  Infinite(τ), start = τ -> 0.0)
    @variable(model, vx, Infinite(τ), start = τ -> 0.0)
    @variable(model, vy, Infinite(τ), start = τ -> 0.0)
    @variable(model, vz, Infinite(τ), start = τ -> 0.0)

    # ONLY in-plane control: ux and uy allowed, uz = 0 fixed
    @variable(model, -u_max <= ux <= u_max, Infinite(τ))
    @variable(model, -u_max <= uy <= u_max, Infinite(τ))
    @variable(model, uz, Infinite(τ), start = 0.0)  # exists only for completeness
    fix.(uz, 0.0; force = true)                      # HARD FIX: no out-of-plane thrust

    # Potential gradients (full 3D)
    r1(x_, y_, z_) = sqrt((x_ + μ)^2 + y_^2 + z_^2)
    r2(x_, y_, z_) = sqrt((x_ - (1 - μ))^2 + y_^2 + z_^2)
    ∂Ωx(x_, y_, z_) = x_ - (1 - μ) * (x_ + μ) / r1(x_, y_, z_)^3 -
                          μ * (x_ - (1 - μ)) / r2(x_, y_, z_)^3
    ∂Ωy(x_, y_, z_) = y_ - (1 - μ) * y_ / r1(x_, y_, z_)^3 -
```

```
                            μ * y_ / r2(x_, y_, z_)^3
    ∂Ωz(x_, y_, z_) = -(1 - μ) * z_ / r1(x_, y_, z_)^3 -
                            μ * z_ / r2(x_, y_, z_)^3

    model[:∂Ωx] = ∂Ωx
    model[:∂Ωy] = ∂Ωy
    model[:∂Ωz] = ∂Ωz
    model
end

### Cell 5: Dynamics + BCs + Objective (uz = 0 enforced, MINIMUM ENERGY)
```

$$tf \times \int_{\tau \in [0,1]} ux(\tau)^2 + uy(\tau)^2 d\tau$$

```
begin
    # Full 3D dynamics
    @constraint(model, ∂(x,  τ) == tf * vx)
    @constraint(model, ∂(y,  τ) == tf * vy)
    @constraint(model, ∂(z,  τ) == tf * vz)
    @constraint(model, ∂(vx, τ) == tf * (2 * vy  + model[:∂Ωx](x, y, z) + ux))
    @constraint(model, ∂(vy, τ) == tf * (-2 * vx + model[:∂Ωy](x, y, z) + uy))
    @constraint(model, ∂(vz, τ) == tf * (model[:∂Ωz](x, y, z) + uz))  # uz = 0 → pure gravity in z

    # Initial: parked near Earth, zero velocity, in xy-plane
    @constraint(model, x(0)  == 1 - μ + 0.001)
    @constraint(model, y(0)  == 0.0)
    @constraint(model, z(0)  == 0.0)
    @constraint(model, vx(0) == 0.0)
    @constraint(model, vy(0) == 0.0)
    @constraint(model, vz(0) == 0.0)

    # Final: halt at L1, zero velocity, stay in plane
    @constraint(model, x(1)  == x_L1)
    @constraint(model, y(1)  == 0.0)
    @constraint(model, z(1)  == 0.0)
    @constraint(model, vx(1) == 0.0)
    @constraint(model, vy(1) == 0.0)
    @constraint(model, vz(1) == 0.0)

    # ── MINIMUM ENERGY OBJECTIVE ──────────────────────────────────────
    # Minimise the physical energy integral: tf * ∫₀¹ (ux² + uy²) dτ
    # The tf scaling converts the normalized-time integral to physical time,
    # preventing the solver from artificially extending tf to reduce cost.
    # Only in-plane controls enter the cost since uz ≡ 0.
    @objective(model, Min, tf * ∫(ux^2 + uy^2, τ))
end

### Cell 6: Solve
```

```julia
begin
    println("Starting optimization... (this will take ~60-120 seconds)")
    @time optimize!(model)
    println("Done! Termination status: ", termination_status(model))
end

### Cell 7: Extract solution
```

```
Starting optimization... (this will take ~60-120 seconds)

******************************************************************************
This program contains Ipopt, a library for large-scale nonlinear optimization.
 Ipopt is released as open source code under the Eclipse Public License (EPL).
         For more information visit https://github.com/coin-or/Ipopt
******************************************************************************

This is Ipopt version 3.14.19, running with linear solver MUMPS 5.8.1.

Number of nonzeros in equality constraint Jacobian...:    23040
Number of nonzeros in inequality constraint Jacobian.:        0
Number of nonzeros in Lagrangian Hessian.............:   531061

Total number of variables............................:     7015
                     variables with only lower bounds:        0
                variables with lower and upper bounds:     1003
                     variables with only upper bounds:        0
Total number of equality constraints.................:     6018
Total number of inequality constraints...............:        0
        inequality constraints with only lower bounds:        0
   inequality constraints with lower and upper bounds:        0
        inequality constraints with only upper bounds:        0
```

```
([1.001, 1.13617, 1.31356, 1.44072, 0.341602, -0.162265, -0.135518, 0.300563, 0.945637,    mor
```

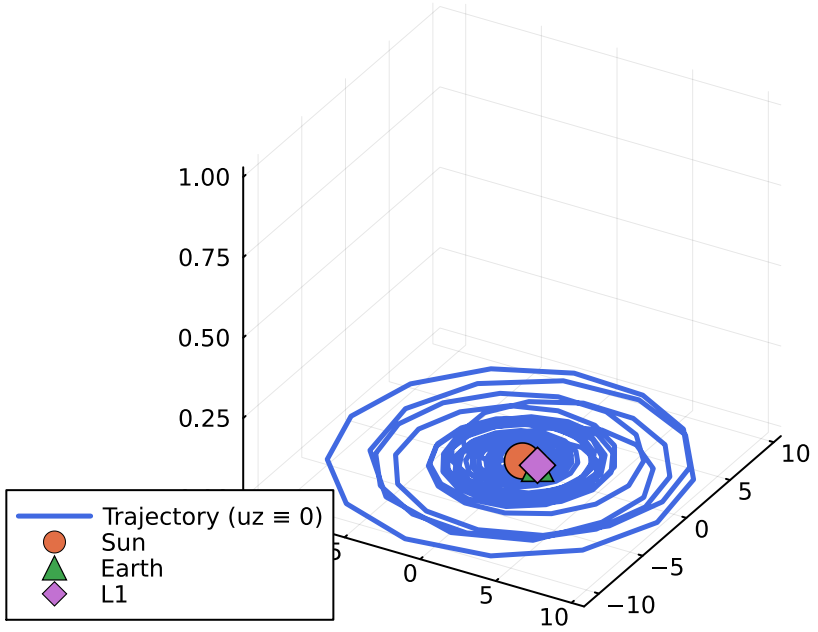
```
begin
    opt_x,  opt_y,  opt_z  = value(x),  value(y),  value(z)
    opt_vx, opt_vy, opt_vz = value(vx), value(vy), value(vz)
    opt_ux, opt_uy         = value(ux), value(uy)
    opt_uz                 = value(uz)   # should be exactly zero everywhere
    opt_tf                 = value(tf)
    τ_grid  = value.(supports(τ))
    t_grid  = τ_grid .* opt_tf

    println("Transfer time: ", round(opt_tf, digits = 2), " normalized units")
    println("             ≈ ", round(opt_tf * 58.1328, digits = 1), " days")
    println("             ≈ ", round(opt_tf * 58.1328 / 365.25, digits = 2), " years")
    println("Max |z|:  ", maximum(abs, opt_z),  " (should be < 1e-8)")
    println("Max |uz|: ", maximum(abs, opt_uz))

    # Energy cost
    energy = opt_tf * sum((opt_ux .^ 2 .+ opt_uy .^ 2) .* diff([t_grid; t_grid[end]]))
    println("Approximate energy cost (tf·∫(ux²+uy²)dt): ", round(energy, digits = 6))

    solution = (opt_x, opt_y, opt_z, opt_vx, opt_vy, opt_vz,
                opt_ux, opt_uy, opt_uz, t_grid, opt_tf)
end

### Cell 8: Plots
```

```
Transfer time: 230.82 normalized units
             ≈ 13418.3 days
             ≈ 36.74 years
Max |z|:  0.0 (should be < 1e-8)
Max |uz|: 0.0
Approximate energy cost (tf·∫(ux²+uy²)dt): 1046.021424
```

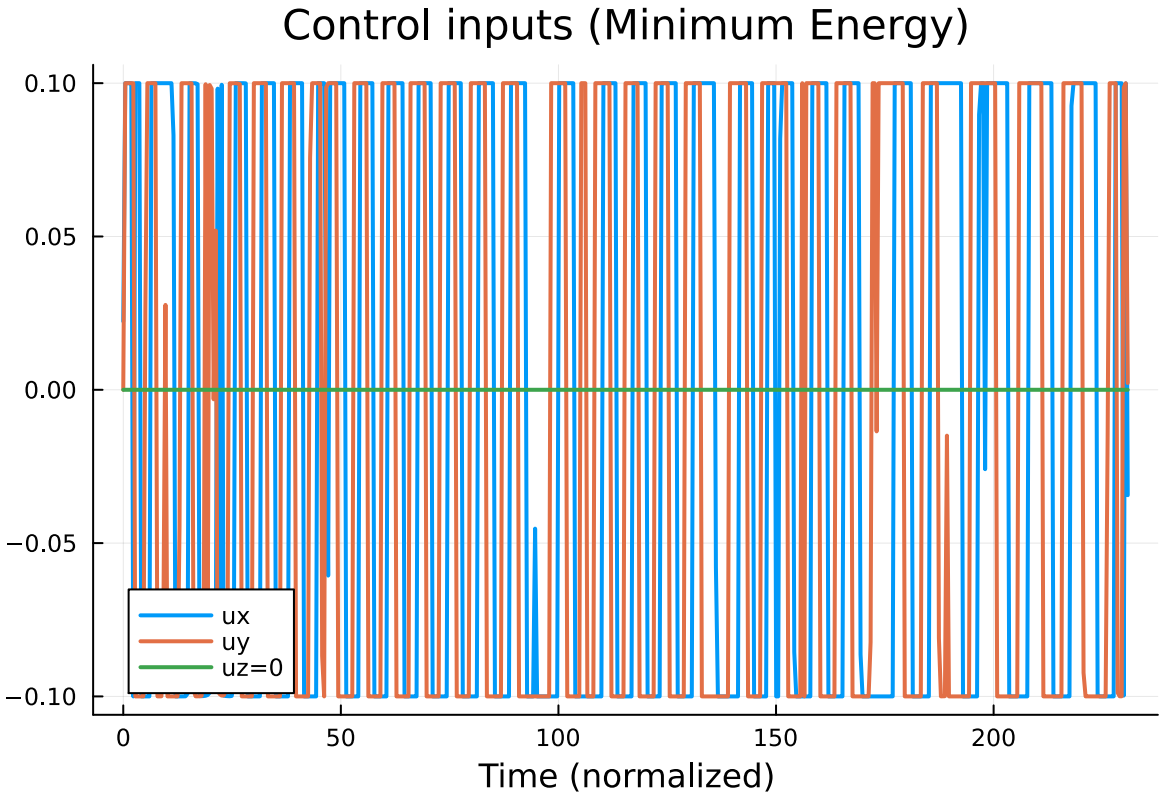
```
begin
    # 3D trajectory
    p_traj = plot(opt_x, opt_y, opt_z,
        lw    = 2.5,
        label = "Trajectory (uz ≡ 0)",
        title = "Earth → Sun-Earth L1 with ONLY in-plane thrust (Min Energy)",
        color = :royalblue)
    scatter!(p_traj, [0.0],   [0.0], [0.0],
        marker = :circle,   markersize = 10, label = "Sun")
    scatter!(p_traj, [1 - μ], [0.0], [0.0],
        marker = :utriangle, markersize = 8,  label = "Earth")
    scatter!(p_traj, [x_L1],  [0.0], [0.0],
        marker = :diamond,  markersize = 9,  label = "L1")

    # xy-plane projection
    p_xy = plot(opt_x, opt_y,
        aspect_ratio = :equal,
        label        = false,
        title        = "xy-plane projection",
        xlabel       = "x", ylabel = "y")

    # xz-plane — should be a flat line at z = 0
    p_xz = plot(opt_x, opt_z,
        label  = false,
        title  = "xz-plane (should be flat line)",
        xlabel = "x", ylabel = "z")

    # Control inputs
    p_ctrl = plot(t_grid, [opt_ux opt_uy opt_uz],
        label  = ["ux" "uy" "uz=0"],
        lw     = 2,
        title  = "Control inputs (Minimum Energy)",
        xlabel = "Time (normalized)")

    display(p_traj)
    display(plot(p_xy, p_xz, layout = (1, 2), size = (900, 400)))
    display(p_ctrl)
end
```

```
Plot{Plots.GRBackend() n=4}
Plot{Plots.GRBackend() n=2}
Plot{Plots.GRBackend() n=3}
```

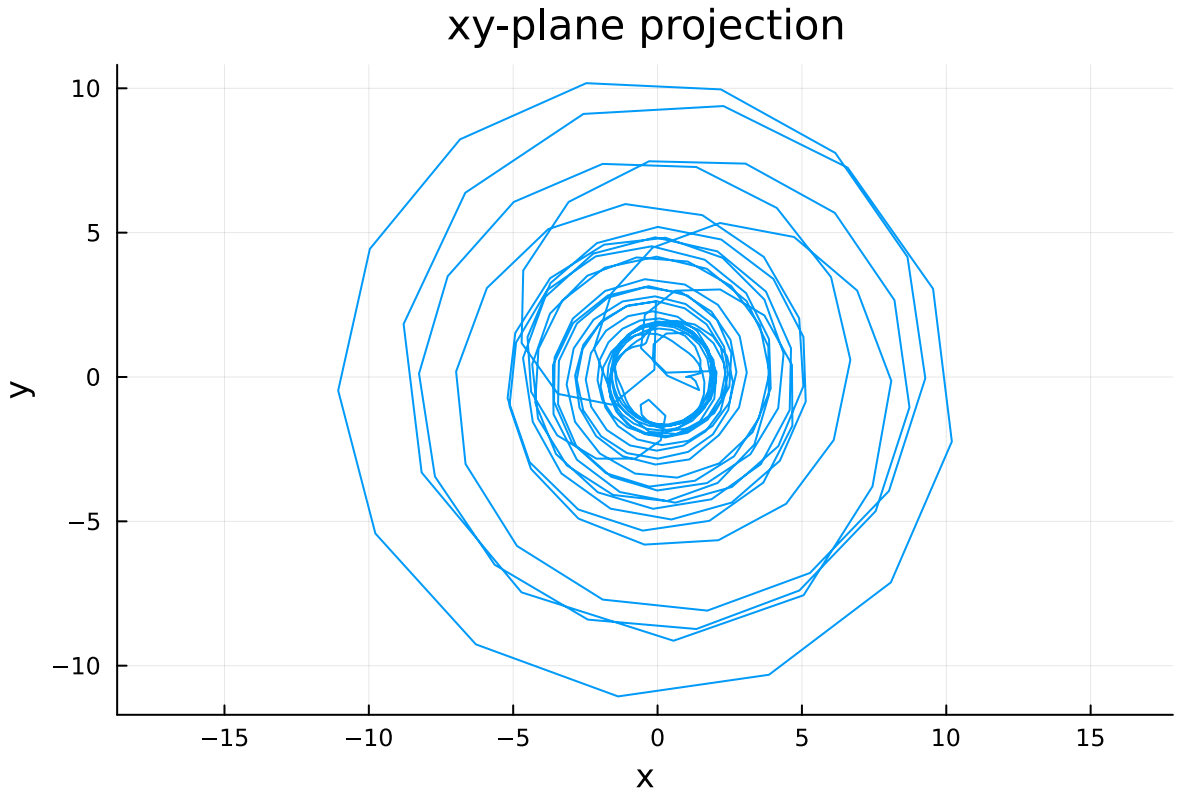
# Earth → Sun–Earth L1 with ONLY in-plane thrust (Min Ener



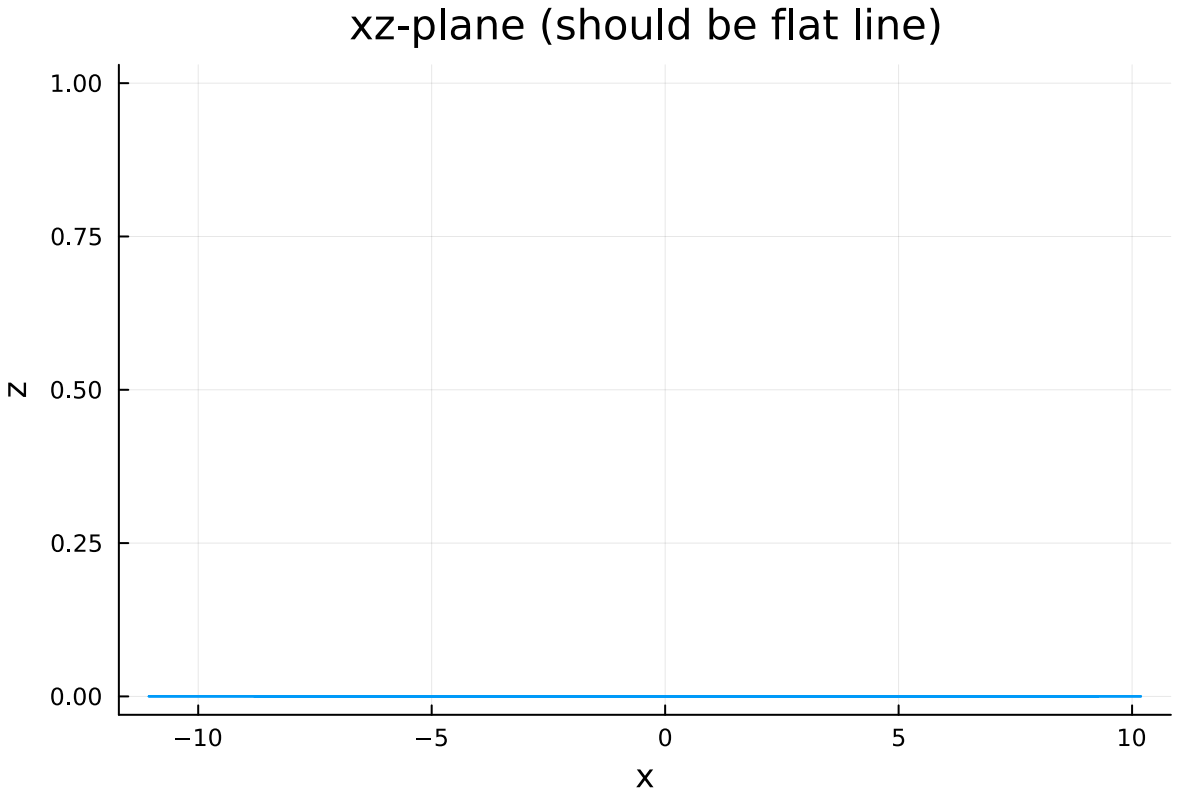

```
1 p_traj
```


Control inputs (Minimum Energy)
0.10
0.05
0.00
−0.05
−0.10
0
50
100
150
200
Time (normalized)
ux
uy
uz=0

```
1 p_ctrl
```




```
1 p_xy
```




```
1 p_xz
```